

Attributes reflection

Document #: [P3385R0](#)
Date: 2024-09-04
Project: Programming Language C++
Audience: sg7
Reply-to: Aurelien Cassagnes
<aurelien.cassagnes@gmail.com>
Roman Khoroshikh
<rkhoroshikh@bloomberg.net>
Anders Johansson
<ajohansson12@bloomberg.net>

Contents

- [1 Introduction](#)
 - [1.1 Earlier work](#)
 - [1.2 Scope](#)
 - [1.3 Optionality rule](#)
- [2 Proposed Features](#)
 - [2.1 std::meta::info](#)
 - [2.2 Reflection operator](#)
 - [2.3 Splicers](#)
 - [2.4 Metafunctions](#)
 - [2.5 Queries](#)
- [3 Proposed wording](#)
 - [3.1 Language](#)
 - [3.2 Library](#)
- [4 Discussion](#)
- [5 References](#)

§ 1 Introduction

Attributes are used to great extent and there likely will be attributes added as the language evolves.

What is missing as reflection makes its way into our standard, is a way for generic code to look into the attributes appertaining to an entity. This is what the proposal here aims to tackle.

A motivating example is the following

```
enum class Result {
    success,
    warn,
    fail,
};

struct [[nodiscard]] StrictNormalize {
    static constexpr bool operator() (Result status) {
        return status == Result::success;
    }
};

template <class F>
[[ [[: ^F :] ]] // expand into [[nodiscard]]
auto transform(auto... args) {
    return F()(args...);
};

int main() {
    transform<StrictNormalize>(Result::success); // warning on
    "nodiscard"
    bool isOk = transform<StrictNormalize>(Result::success); // OK
}
```

Here transform looks into the attributes attached to the callable F, and recovers the `[[nodiscard]]` attribute that appertained originally to StrictNormalize declaration.

We expect number of applications for attribute introspection to happen in the context of code injection [P2237R0] where one may want to skip over `[[deprecated]]` members for example. Following example demonstrates skipping over deprecated members

```
struct User {
    [[deprecated]] std::string name;
    std::string uuidv5;
    [[deprecated]] std::string country;
    std::string countryIsoCode;
};

template<class T>
constexpr std::vector<std::meta::info> liveMembers(const T& user) {
    std::vector<std::meta::info> liveMembers;
    auto deprecatedAttribute = std::meta::attributes_of(^[[deprecated]])
        [0];
    auto keepLive = [&] <auto r> {
        if (!std::ranges::any_of(
```

```

        attributes_of(^T),
        [deprecatedAttribute] (auto meta) { meta == deprecatedAttributes;
    }
    )) {
        liveMembers.push_back(r);
    }
};

template for (auto member : std::meta::members_of(^User)) {
    keepLive(member);
}
return os;
}

// Migrated user will no longer support deprecated fields
struct MigratedUser;
define_class(^MigratedUser, liveMembers(currentUser));

```

§ 1.1 Earlier work

While collecting feedback on this draft, we were redirected to [P1887R1] as a pre existing proposal. In this paper the author discusses two topics ‘user defined attributes’ (also in [P2565R0]) and reflection over said attributes. We believe the two topics need not be conflated together, both have intrinsic values on their own. We aim here to focus the discussion entirely on **standard** attributes reflection. Furthermore this earlier paper has not seen work following the progression of [P2996R4] and so we feel this proposal is in a good place to fill that gap.

§ 1.2 Scope

Attributes are split into standard and non standard. This proposal wishes to limit itself to standard attributes (9.12 [dcl.attr]). We feel that since it is up to implementation to define how they handle non standard attributes, it would lead to obscure situations that we don’t claim to tackle here.

A fairly (admittedly artificial) example can be built as such: Given an implementation supporting a non standard `[[no_introspect]]` attributes that suppress all reflection information appertaining to an entity, we would have a hard time coming up with a self-consistent system of rules to start with.

Henceforth in this proposal, both ‘attributes’ and ‘standard attributes’ terms are meant to be equivalent.

§ 1.3 Optionality rule

There is a long standing and confusing discussion around the ignorability of attributes. We’ll refer the reader to [P2552R3] for an at-length discussion of this problem, and especially on what ‘ignorability’ really means. This proposal agrees with the discussion

carried there and in [CWG2538]. We also feel that whether an implementation decides to semantically ignore a standard attribute should not matter.

What matters more is self-consistency, when introspecting an entity:

- We should be able to discover appertaining attributes
- Declaring an entity with attributes discovered via introspection yield the same result as what the implementation would offer if directly declaring that entity with those attributes.

§ 2 Proposed Features

We put ourselves in the context of [P2996R4] for this proposal to be more illustrative in terms of what is being proposed.

§ 2.1 `std::meta::info`

We propose that attributes be a supported *reflectable* property of the expression that are reflected upon. That means value of type `std::meta::info` should be able to represent an attribute in addition to the currently supported set.

§ 2.2 Reflection operator

The current proposition for reflection operator grammar does not cover attributes, i.e. the expression `^[[deprecated]]` is ill-formed. Our proposal advocates to support expression as following

```
constexpr auto keepAttribute = ^[[nodiscard]];
```

The resulting value is a reflection value embedding relevant info for the *attribute* entity, in this case the `nodiscard` attribute.

If the *attribute* is not a standard attribute, the expression is ill-formed.

§ 2.3 Splicers

We propose that the syntax

```
[[ [: r :] ]]
```

be supported in contexts where attributes are allowed.

- `[[ [: r :] ]]` produces a potentially empty *attribute-list* corresponding to the attributes reflected via reflection `r`.

Note that as it stands now *attribute-list* (9.12.1 [dcl.attr.grammar]) does not cover `alignas`. We understand that this limits potential use of the current proposal but also comes with difficulty, so this will be discussed in a separate paper.

An artificial example of splicer use using expansion statements is as follows. We create an augmented `enum` introducing a `begin` and `last` enumerator while preserving the original attributes

```
[[nodiscard]]
enum class ErrorCode {
    warn,
    fatal,
};

[[ [ : ^ErrorCode : ] ]]
enum class ClosedErrorCode {
    begin,
    template for (constexpr auto e :
        std::meta::enumerators_of(^ErrorCode)) {
        return [ :e:],
    }
    last,
};
```

If the attributes produced through introspection violate the rules of what attributes can appertain to what entity, as usual the program is ill-formed.

§ 2.3.1 Attribute using

It is worth pointing out the interaction between *attribute-using-prefix* and splice expression that could lead to unexpected results.

In the following example

```
auto unscopedAttribute = ^[[nodiscard]];
[[ using CC: debug, [ : unscopedAttribute : ] ]] enum class Code {};
```

While the user likely does not intend for the standard attributes to be targeted by `using` `CC`, current grammar says that the prefix applies to attributes as they are found in the subsequent list. To remediate this we can either enforce that *splice-name-qualifier* precedes *attribute-using-prefix* or have those constructs be mutually exclusive as they occur in `[[ ]]`.

To reduce the need to memorize unintuitive rules, we favor the later of those options, as following

```
auto unscopedAttribute = ^[[nodiscard]];
[[ [ : unscopedAttribute : ] ]][[ using CC: debug ]] enum class Code {};
```

§ 2.4 Metafunctions

We propose to add two metafunctions to what is discussed already in [P2996R4]. Additionally we will add support for attributes in the other metafunctions where it makes sense.

§ 2.4.1 attributes_of

```
namespace std::meta {  
    consteval auto attributes_of(info entity) -> vector<info>;  
}
```

This being applied to a reflection entity will yield a sequence of `std::meta::info` representing each individual attribute appertaining to entity.

§ 2.4.2 is_attribute

```
namespace std::meta {  
    consteval auto is_attribute(info entity) -> bool;  
}
```

This would return true if the entity reflection designates a `[ [ attribute ] ]`, otherwise it would return false.

§ 2.4.3 name_of, display_name_of

Given a reflection `r` designating a standard attribute, `name_of(r)` (resp. `u8name_of(r)`) is encouraged to return a `string_view` (resp. `u8string_view`) corresponding to the attribute-token. The same holds for the `qualified_name_of` (resp. `u8qualified_name_of`).

A toy example follows

```
[[nodiscard, deprecated("Do not use me")]] int func();  
  
void print_attributes(std::meta I) {  
    template for (bool first{true}; constexpr auto e :  
        std::meta::attributes_of(I)) {  
        std::cout << (std::exchange(first, false) ? "" : ", ")  
            << std::meta::name_of(e) << std::endl;  
    }  
}  
  
print_attributes(^func); // Prints "nodiscard, deprecated"
```

Given a reflection `r` that designates an individual attribute, `display_name_of(r)` (resp. `u8display_name_of(r)`) returns an unspecified non-empty `string_view` (resp.

u8string_view). Implementations are encouraged to produce text that is helpful in identifying the reflected attribute.

§ 2.4.4 data_member_spec, define_class

As it stands now define_class allows piecewise building of a class via data_member_spec. To support attributes pertaining to those data members however, we'll need to augment data_member_options_t to encode attributes we may want to attach to a data member.

The structure will change thusly

```
namespace std::meta {
    struct data_member_options_t {
        struct name_type {
            template <typename T> requires
                constructible_from<u8string, T>
                consteval name_type(T &&);

            template <typename T> requires
                constructible_from<string, T>
                consteval name_type(T &&);
        };

        optional<name_type> name;
        bool is_static = false;
        optional<int> alignment;
        optional<int> width;
+       vector<info> attributes;
    };
}
```

From there building a class piecewise proceeds as usual

```
struct Empty {};
struct [[nodiscard]] S;
define_class(^S, {
    data_member_spec(^int, {.name = "i"}),
    data_member_spec(^Empty, {.name = "e",
                             .attributes = {^[[no_unique_address]]} })
});

// Equivalent to
// struct [[nodiscard]] S {
//     int i;
//     [[no_unique_address]] Empty e;
// };

```

§ 2.4.5 Other metafunctions

For any reflection where `is_attribute` returns false, other metafunctions not listed above are not considered constant expressions

§ 2.5 Queries

We do not think it is necessary to introduce additional query or queries at this point. Especially we would not recommend to introduce a dedicated query per attribute (eg `is_deprecated`, `is_nouniqueaddress`, etc.). Having said that, we feel those should be achievable via concepts, something akin to

```
auto deprecatedAttributes = std::meta::attributes_of(^[[deprecated]]);

template<class T>
concept IsDeprecated = std::ranges::any_of(
    attributes_of(^T),
    [deprecatedAttributes] (auto meta) { meta ==
    deprecatedAttributes[0]; }
);
```

§ 3 Proposed wording

§ 3.1 Language

§ 3.1.1 [dcl.attr.grammar] Attribute syntax and semantics

Change the grammar to allow splicing attributes from reflection in 9.12.1 [dcl.attr.grammar] as follows

attribute-specifier:

$$[[\textit{attribute-using-prefix}_{opt} \textit{attribute-list}]]$$
$$[[\textit{splice-name-qualifier}]]$$

Following this subsection modify the paragraph 7

- 7 Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier* ~~or~~, within the *balanced-token-seq* of an *attribute-argument-clause* or as part of a *reflect-expression*.

Add the following paragraph

- 8 If an *attribute-specifier* contains a *splice-name-qualifier*, every standard attribute described by that reflection must be applied to the entity that *attribute-specifier* is attached to. The form of this expansion shall be treated as an *attribute-list*. If *splice-name-qualifier* describes an entity for which no attributes were specified, it has no effect.

§ 3.1.2 [expr.unary.general] General

Augment the `^` operator to allow for reflection on attribute in 7.6.2.1 [expr.unary.general] as follows

reflect-expression:

```
^ ::  
^ namespace-name  
^ nested-name-specifieropt template-name  
^ nested-name-specifieropt concept-name  
^ type-id  
^ id-expression  
^ [ [ attribute ] ]
```

§ 3.1.3 [expr.reflect] The reflection operator

Modify the subsection [expr.reflect] to describe reflection applying to attributes, adding the following paragraph

- ¹¹ When applied to a [[attribute]], the reflection operator produces a reflection for the indicated *attribute*. If *attribute* is not described in 9.12 [dcl.attr], the behavior is implementation defined.

§ 3.2 Library

§ 3.2.1 [meta.synop] Header `<meta>` synopsis

Add to the [meta.reflection.queries] section from the synopsis the two metafunctions as follow

```
constexpr bool is_attribute(info r);  
constexpr vector<info> attributes_of(info r);
```

§ 3.2.2 [meta.reflection.queries], Reflection queries

- ⁴² `constexpr bool is_attribute(info r);`

Returns: true if *r* designates a standard attribute. Otherwise, false.

```
constexpr vector<info> attributes_of(info r);
```

Returns: A vector containing a reflection for each of the attributes that appertain to the entity *r*

§ 4 Discussion

Originally the idea of introducing a `declattr(Expression)` keyword seemed the most straightforward to tackle this problem, but from feedback the concern of introspecting on expression attributes was a concern that belongs with the reflection SG. The current proposal shifted away from the original `declattr` idea to align better with the reflection toolbox. Note also that as we advocate here for `[[[: r :]]]` to be supported, we recover the ease of use that we first envisioned `declattr` to have.

§ 5 References

- [CWG2538] Jens Maurer. 2021-12-02. Can standard attributes be syntactically ignored?
<https://wg21.link/cwg2538>
- [P1887R1] Corentin Jabot. 2020-01-13. Reflection on attributes.
<https://wg21.link/p1887r1>
- [P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
<https://wg21.link/p2237r0>
- [P2552R3] Timur Doumler. 2023-06-14. On the ignorability of standard attributes.
<https://wg21.link/p2552r3>
- [P2565R0] Bret Brown. 2022-03-16. Supporting User-Defined Attributes.
<https://wg21.link/p2565r0>
- [P2996R4] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-06-26. Reflection for C++26.
<https://wg21.link/p2996r4>